TASK 4

Text classification

You are in the group Mlpeleton consisting of iturner (iturner@student.ethz.ch (mailto://iturner@student.ethz.ch)), lmaloney (lmaloney@student.ethz.ch (mailto://lmaloney@student.ethz.ch)) and ssambale (ssambale@student.ethz.ch (mailto://ssambale@student.ethz.ch)).

1. READ THE TASK DESCRIPTION

2. SUBMIT SOLUTIONS

3. HAND IN FINAL SOLUTION

1. TASK DESCRIPTION

GENERATING SCORES FROM USER REVIEWS

Welcome to task #4, the last project of this year's Introduction to Machine Learning course at ETH!

When customers shop online, they can leave reviews for the purchased product. In this task, given a dataset containing the review's title and its content, you will try to infer whether the review is positive or negative.

DATA DESCRIPTION

You can download the project material here:

Download (/static/task4_hr35z9.zip) (4.4 MB)

In the handout, you will find the following material:

- **train.csv** - a csv containing the columns "title", "sentence", and "score". The score is either positive (1), or negative (0).
- **test_no_score.csv** - csv containing the columns "title" and "sentence".
- **sample.txt** - a sample submission file.
- **template_solution.py** - a template file that will guide you through the implementation of the solution.
- **template_solution.ipynb** - the template solution in an interactive notebook form.

template_solution.py provides a starting template structure for how you can solve the task, by filling in the TODOs in the skeleton code. It is not mandatory to use this solution template but it is recommended since it should make getting started on the task easier. The template solution uses the PyTorch (https://pytorch.org/) deep learning framework.

SUGGESTED APPROACH

The recommended approach is to use the transformers (https://huggingface.co/docs/transformers/index) library, which contains the necessary network architectures and pretrained transformers. It is recommended to look into the following transformers:

- DistilBERT (https://huggingface.co/docs/transformers/en/model_doc/distilbert#transformers.DistilBertModel)
- ALBERT (https://huggingface.co/docs/transformers/en/model_doc/albert#transformers.AlbertModel)
- GPT-2 (https://huggingface.co/docs/transformers/en/model_doc/gpt2#transformers.GPT2Model)

Note that while traditional approaches using pretrained transformers requires fine-tuning them for the desired task, it is **NOT** required to fine-tune the transformers to pass the baseline. Using frozen weights is sufficient to pass the hard baseline. To do so, one can run the transformer separately and save the results to a standalone file. When needed, the results are loaded without additional computation. It is recommended to refer to previous tasks for examples on how this is done. Interested students with sufficient computational resources are encouraged to fine-tune the models. This best mimics real world use cases of pretrained transformers.

VOLUNTARY TASK

Interested students are encouraged to explore the challenge of training transformer models under memory and compute constraints. As an extra incentive, a prize will be awarded at the end of the semester for the most creative or interesting solution.

The setting assumes a resource-constrained environment where full fine-tuning is not feasible. Training large models with limited resources is an active area of research, and there is no single best solution — the goal is to find, combine, or even devise methods that achieve the best possible performance under these constraints.

Students are encouraged to explore techniques such as:

- Quantization (https://huggingface.co/docs/transformers//main_classes/quantization) — reducing numerical precision to lower memory footprint
- Parameter-Efficient Fine-Tuning (PEFT) methods, e.g. LoRA (https://arxiv.org/abs/2106.09685) and GaLore (https://arxiv.org/abs/2403.03507)
- Neural network pruning to reduce model size and inference cost: link1 (https://arxiv.org/abs/2102.00554), link2 (https://arxiv.org/abs/2301.00774)

Creative combinations of the above, or entirely novel approaches, are equally welcome.

Due to the heavy computational cost associated with transformer inference, it is recommended to use a modern GPU for this task. Students are invited to use Google Colab or the ETH Euler cluster. If no GPU can be obtained, it is possible to pass the hard baseline using CPU-based computation on the ETH Euler cluster within a reasonable time limit.

EVALUATION

We are interested in correctly classifying each review as positive or negative. Your submitted predictions will be evaluated by the *accuracy* against the true labels. Higher is better. Accuracy is defined as

$$\text{Accuracy}(\mathbf{y}^*, \mathbf{y}) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}_{\{y_i^* = y_i\}}$$

SUGGESTED MATERIAL (OPTIONAL)

You will learn all the details about transformers in the last few lectures of this course. However, if you are interested about the inner workings of transformer models the following will prove useful.

- Last year's IML material on transformers.

    - Lecture slides. (https://las.inf.ethz.ch/courses/introml-s23/slides/introml-24-transformers.pdf)
    - Lecture recording (https://video.ethz.ch/lectures/d-infk/2023/spring/252-0220-00L/530d20a9-d02a-45e1-a141-42c54d510eae.html)
    - Tutorial recordings (https://video.ethz.ch/lectures/d-infk/2023/spring/252-0220-00L/82638d96-1686-404a-9e11-ee01a7ed5595.html)
- Hugging Face tutorials on transformers. (https://huggingface.co/docs/transformers/index)
- 3Blue1Brown. But what is a GPT? Visual intro to transformers. (https://www.youtube.com/watch?v=wjZofJX0v4M)
- Attention is all you need. (https://arxiv.org/abs/1706.03762)

GRADING

This is a pass — fail project and you will not receive a grade for this task. You need to pass 3 / 4 projects in the IML course to be eligible for taking the exam. Your algorithm will make predictions on a held out test set. For each of your submissions, a score is computed.

When handing in the task, you need to select which of your submissions will get graded and provide a public link to a maximum one minute long video explaining your approach. Make sure that we can access the link till the first of October. Every student has to submit an individual 1 minute long video. This has to be done **individually by each member** of the team. For generating the video link, please use the ETH polybox (https://polybox.ethz.ch/index.php/login/) service, upload your video there and generate a shareable link (see this documentation (https://polyboxdoc.ethz.ch/share-with-people-outside-of-eth/) for more detail). Submissions that are not handed-in, do not have a video that we can access till the first of October or have a video exceeding the one minute threshold will not be graded.

We will compare your selected submission to our public baseline, which is visible on the public leaderboard. **You pass this project if your submitted solution beats our public baseline.** At the end of the semester, we will award one team with a certificate and prize for their performance on this task (excluding task 0). The selection criteria are based on the team's performance on our public and private leaderboards and the creativity of their solution. The prize will be disclosed at the end of the semester. The private leaderboards are based on a separate test score on an undisclosed test set. You only receive feedback about your performance on the public part in the form of the public score, while the private leaderboard remains secret. The purpose of this division is to prevent overfitting to the public score. Your model should generalize well to the private part of the test set. The creativity of your solution will be evaluated by our TAs based on your video submission.

⚠ Make sure that you properly hand in the task, otherwise you will fail this task.

PLAGIARISM

The use of open-source libraries is allowed and encouraged. However, we do not allow copying the work of other groups / students outside the group (including work produced by students in previous versions of this course). Publishing project solutions online is not allowed and use of solutions from previous years in any capacity is considered plagiarism. Among the code and the reports, including those of previous years, we search for similar solutions / reports in order to detect plagiarism. Although not strictly forbidden, we discourage the use of Github Copilot or similar code/language generation tools for writing code. We expect that if such tools are used, this is clearly stated in the video submission explaining the solution. While it will have no effect on your grade or if a solution passes or fails, it may affect the awarding of prizes for best solutions. We discourage these tools because we feel that the best way to understand the material is to write the code yourself referring to just the lecture material, source papers and documentation of any libraries used. For the purposes of disclosing what generative AI tools you used to write code, we don't need you to disclose using e.g. basic code autocompletion such as those used in the default setup of Sublime Text 3. If we find strong evidence for plagiarism, we reserve the right to let the respective students or the entire group fail in the IML 2025 course and take further disciplinary actions. By submitting the solution, you agree to abide by the plagiarism guidelines of IML 2025.

FREQUENTLY ASKED QUESTIONS

WHICH PROGRAMMING LANGUAGE AM I SUPPOSED TO USE? WHAT TOOLS AM I ALLOWED TO USE?

You are free to choose any programming language and use any software library. However, **we strongly encourage you to use Python**. You can use publicly available code, but you should specify the source as a comment in your code.

WHAT TO DO IF I CAN'T RUN THE CODE/SETUP AN ENVIRONMENT ON MY PC?

If you are having trouble running your solution locally, consider using the ETH Euler cluster to run your solution. Please follow the Euler guide (/static/euler-guide.md). The setup time of using the cluster means that this option is only worth doing if you really can't run your solution locally.

AM I ALLOWED TO USE MODELS THAT WERE NOT TAUGHT IN THE CLASS?

Yes. Nevertheless, the baseline was designed to be solvable based on the material taught in the class up to the second week of each task.

IN WHAT FORMAT SHOULD I SUBMIT THE CODE?

You can submit it as a single file (main.py, etc.; you can compress multiple files into a .zip) having max. size of 1 MB. If you submit a zip, please make sure to name your main file as *main.py* (possibly with other extension corresponding to your chosen programming language).

WILL YOU CHECK / RUN MY CODE?

We will check your code and compare it with other submissions. We also reserve the right to run your code. Please make sure that your code is runnable and your predictions are reproducible (fix the random seeds, etc.). Provide a readme if necessary (e.g., for installing additional libraries).

SHOULD I INCLUDE THE DATA IN THE SUBMISSION?

No. You can assume the data will be available under the path that you specify in your code.

CAN YOU HELP ME SOLVE THE TASK? CAN YOU GIVE ME A HINT?

As the tasks are a graded (pass/fail) part of the class, **we cannot help you solve them**. However, feel free to ask general questions about the course material during or after the exercise sessions.

CAN YOU GIVE ME A DEADLINE EXTENSION?

> ⚠ We do not grant any deadline extensions!

CAN I POST ON MOODLE AS SOON AS I HAVE A QUESTION?

This is highly discouraged. Remember that collaboration with other teams is prohibited. Instead,

- Read the details of the task thoroughly.
- Review the frequently asked questions.
- If there is another team that solved the task, spend more time thinking.
- Discuss it with your team-mates.

WHEN WILL I RECEIVE THE PRIVATE SCORES? AND THE PROJECT GRADES?

We will publish all grades before the exam the latest.